

OS biglab b

期末汇报

韩佳辛 2026/5/30



CONTENTS

● ArceOS tutorial
分层与基础实验

● 与AI协作的经验积累和流程
搭建: polyglot-harness

● StarryOS 的漏洞修复及功能
改进

● 未来的OS学习展望

五个任务串成一条从教学 OS 到真实应用兼容的路径



01

ArceOS tutorial

分层与基础实验



ArceOS tutorial 先训练的是“沿层次找问题”的能力

练习	主题	app / exercise crates printcolor, hashmap, altalloc, ramfs-rename, sysmap
<code>exercise-printcolor</code>	彩色串口输出	axstd / user-facing API 把接近 std 的接口转成 ArceOS 内核服务调用
<code>exercise-hashmap</code>	<code>HashMap</code> 支持	
<code>exercise-altalloc</code>	bump allocator	axfs / axmm / axtask 文件系统、地址空间、任务等模块化子系统
<code>exercise-ramfs-rename</code>	ramfs <code>rename</code>	HAL / platform / QEMU 串口、页表、设备和虚拟机运行环境
<code>exercise-sysmap</code>	<code>mmap</code> <code>syscall</code>	

Example: ramfs_rename

std::fs::rename
用户调用



axstd
接口适配



axfs::root::rename
VFS 层



RootDirectory::rename
find_best_mount 分发
到具体 FS



axfs_ramfs::DirNode::rename
改 children 的 key

语义

rename 改 name -> node 绑定,
不复制文件内容。

分发

根目录层不转发, 请求到不了
ramfs。

锁

同目录 rename 要避免重复拿不可
重入写锁。因为 spin::RwLock 不可
重入, 如果 remove 和 insert 分别
重复拿同一个目录的写锁, 就可能
死锁。



02

与AI协作的经验积累和流程
搭建: polyglot-harness

polyglot harness 把 AI 协作放进可复现的 OS 工程流程

组件	作用
AGENTS.md	统一规则入口，说明全局流程与强制技能触发条件
skills/*/SKILL.md	可复用 workflow，例如 PR、报告、测试、实验保护
scripts/docker-check.py	Docker preflight，保证构建/测试环境接近 CI
docs/PORTABILITY.md	描述 Claude/Cursor/Codex/Trae 的接入方式
pr-drafts/	保存 PR 草稿，避免把临时报告混入上游 PR



代表性的 skill workflow

experiment-guard

触发

同步 upstream、merge/rebase，或开始会弄脏仓库的实验。

动作

fetch upstream，确认 base branch；冲突立即停止；Docker preflight；记录分支、commit 和验证命令。

OS 实验收益

先固定实验基线，再讨论内核语义，避免在错误分支上调试。

test-authoring

触发

新增或修改 unit/e2e/QEMU/config/regex 测试。

动作

选择 Rust/C/script 落点；补齐 build-*.toml 与 qemu-*.toml；写 success/fail regex。

OS 实验收益

测试证明语义而不是制造 PASS；backtrace 要看 BT/ip，BusyBox 要看 FAIL: 0。

auto-skill-maintainer

触发

发现重复踩坑、流程缺口、错误 skill 指令，或某个验证步骤应该固化下来。

动作

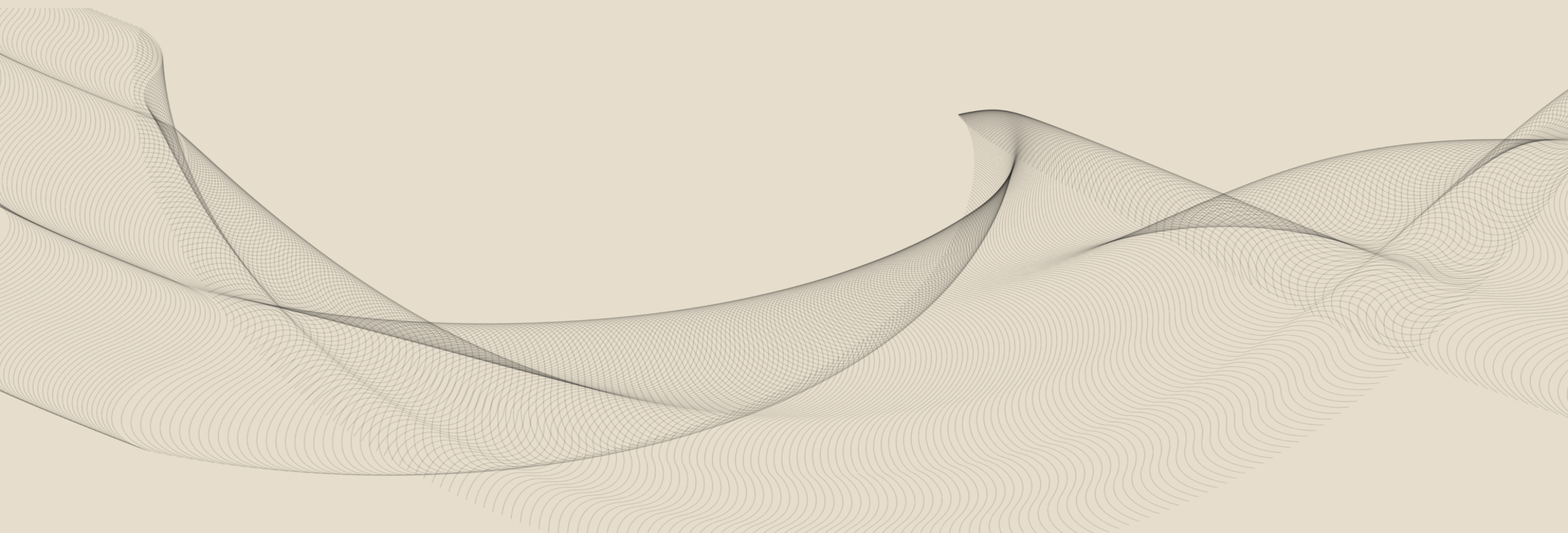
更新或新增 SKILL.md，做最小验证，并把 harness 变更单独提交。

OS 实验收益

把一次调试经验沉淀成下次自动执行的流程，减少 Docker、测试、PR、CI 上反复犯同类错误。

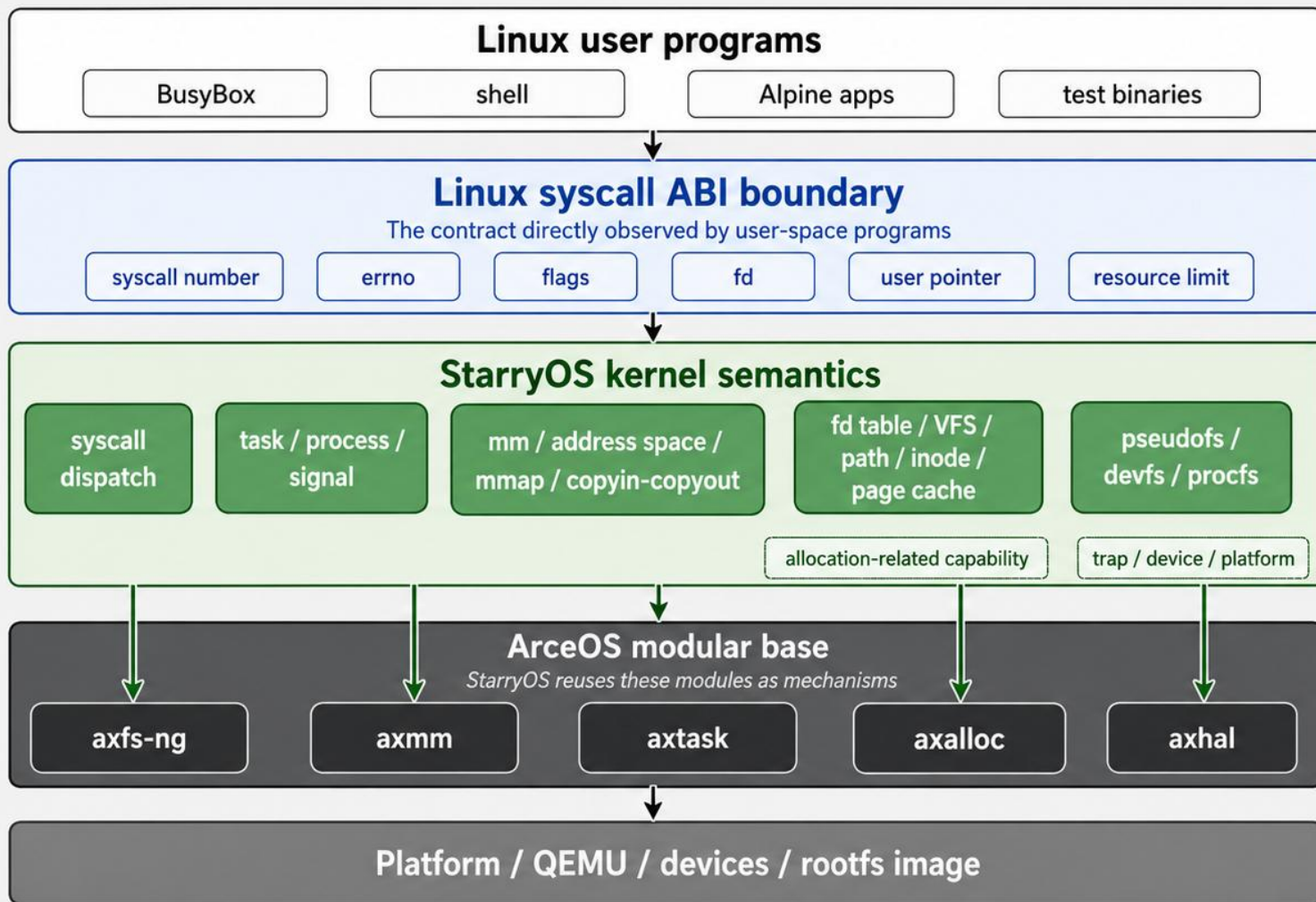
03

StarryOS 的漏洞修复及功能改进



Starry OS 架构分析

StarryOS: Linux 用户态兼容层构建在 ArceOS 模块化底座之上



Why bugs appear at the boundary

- errno priority
- invalid flags side effects
- hard link shared data
- user pointer validation
- BusyBox applet behavior

Starry OS 架构分析

BusyBox / Alpine 程序期望的 Linux 行为



StarryOS 把这些行为翻译成自己的进程、内存、文件、信号等内核对象操作



底层复用 ArceOS 的 axfs / axmm / axtask / axalloc / axhal 等机制

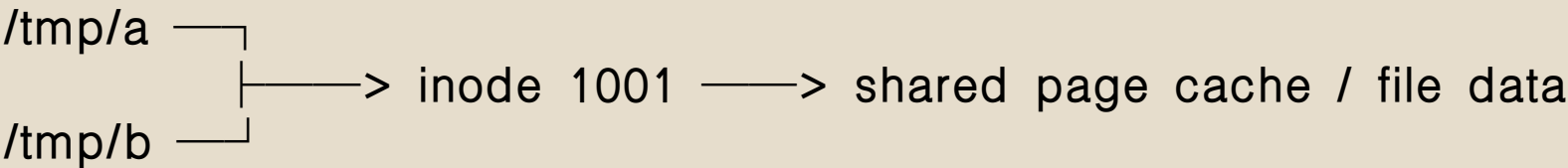


三个 StarryOS PR 都在修用户态可观察的 Linux ABI 细节

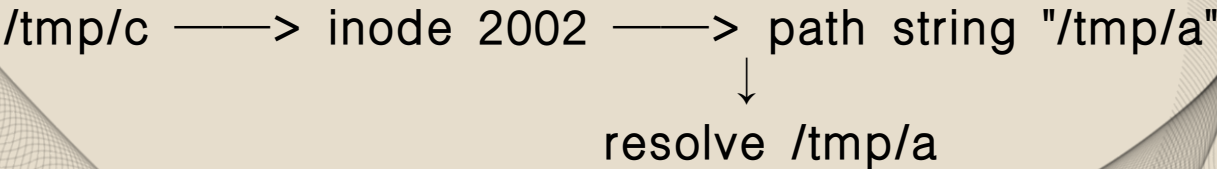
它们的共同点是：错误路径、数据共享和边界组合都不能靠 happy path 推断。

PR	问题	修复点	学到的 OS 语义	状态
#265	unlinkat invalid flags	非法 flags 先返回 EINVAL，不能删除文件	错误路径必须无副作用	merged
#348	tmpfs hardlink 读零	新 DirEntry 继承源 user data	hard link 要共享文件内容状态	merged
#350	边界测试不足	getrandom/prlimit64 等组合输入	错误优先级也是 ABI	merged

Hard link



Soft link



unlinkat 的非法 flags 必须在删除文件之前被拒绝

这是一个很典型的 OS 规则：参数校验失败不能留下副作用。

```
old logic

if flags == AT_REMOVEDIR:
    remove_dir(path)
else:
    remove_file(path)

# invalid bits are ignored
# file may be deleted first
```



```
fixed logic

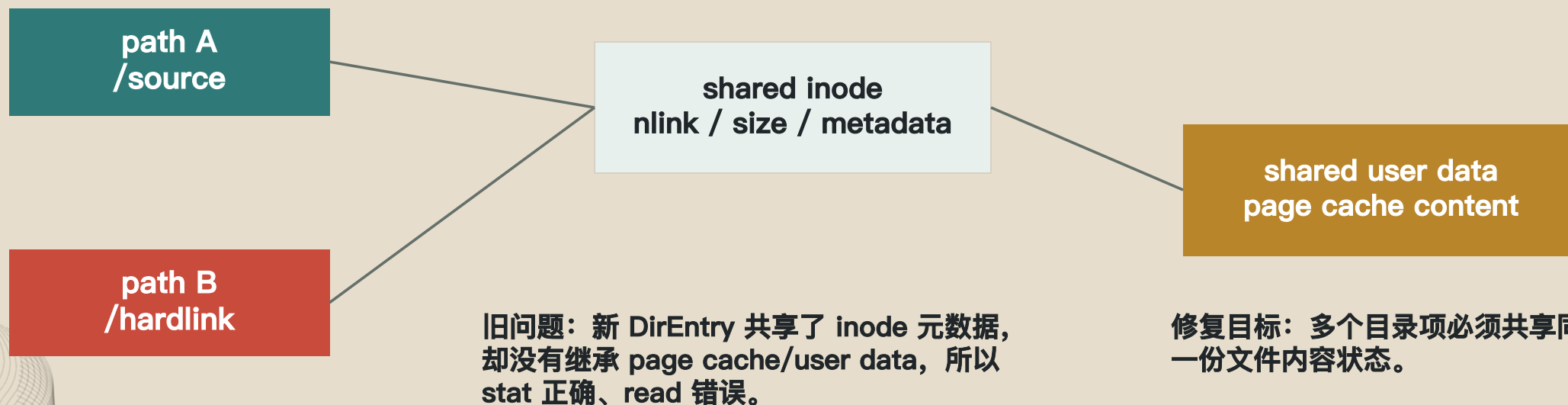
if flags & !AT_REMOVEDIR != 0:
    return EINVAL

if flags & AT_REMOVEDIR != 0:
    remove_dir(path)
else:
    remove_file(path)
```

测试输入	预期	要防止
file + invalid flags	EINVAL, file still exists	先删文件再报错
dir + AT_REMOVEDIR invalid	EINVAL, dir still exists	非法组合被静默接受

tmpfs hardlink 的坑在于 stat 正确不代表数据路径正确

新路径的 inode、nlink、size 都可能对，但 read 仍然读到零填充。

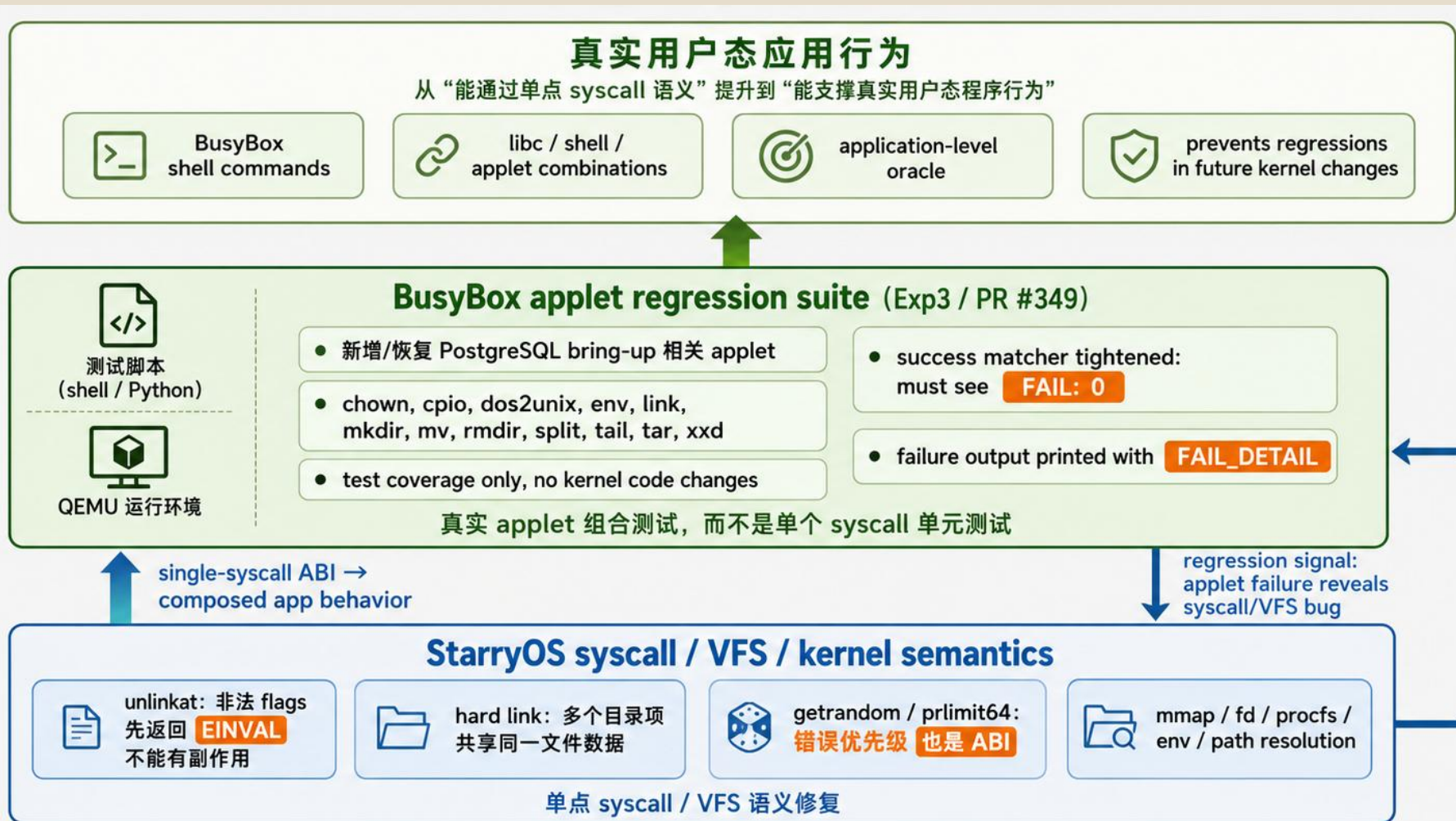


边界测试固定的不只是返回值，还有 Linux errno 的优先级

组合输入的错误顺序是用户态可观察行为，测试必须把它写死。

接口	输入组合	期望	OS 知识
getrandom	len == 0 + bad buffer	返回 0，不访问 buffer	零长度路径不应触碰用户指针
getrandom	invalid flags + bad pointer	优先 EINVAL	错误优先级是 ABI
prlimit64	rlim_cur > rlim_max	EINVAL	resource limit 有内部约束
getrlimit	bad user pointer	EFAULT	copyout 必须校验用户地址

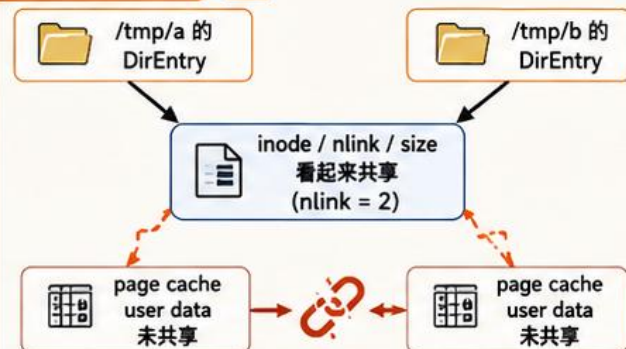
BusyBox 测试的价值在于它验证接口组合，而不是单个函数



BusyBox 把 StarryOS 兼容性从单点 syscall 推向真实 applet 组合路径

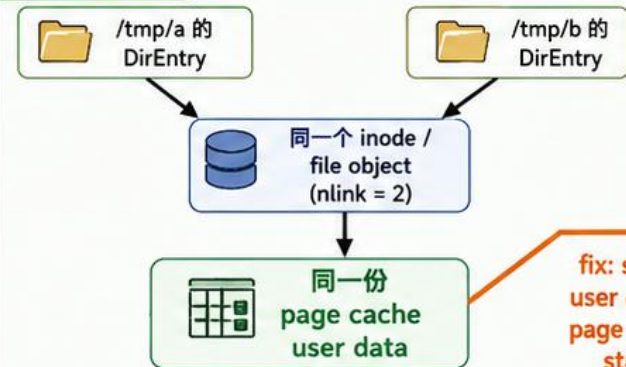
Task2: tmpfs hardlink 修复 (PR #348)

修复前 (BUG)



❌ 结果: cat /tmp/b 读出 0 或错误内容

修复后 (FIX)



✅ 结果: cat /tmp/b == cat /tmp/a

fix: share
user data /
page cache
state

StarryOS hardlink 调用链



Task3: BusyBox applet 回归 (PR #349)

应用级回归测试脚本 (示例)

- 1 busybox rm -f /tmp/bb_link_a /tmp/bb_link_b
- 2 busybox echo link_data > /tmp/bb_link_a
- 3 busybox link /tmp/bb_link_a /tmp/bb_link_b
- 4 busybox cat /tmp/bb_link_b

期望输出: link_data

✅ 测试通过: PASS: busybox_link

🟢 总套件成功条件: FAIL: 0

👤 shell / libc / syscall / VFS 组合路径验证

✅ 真实 applet 行为, 不只是单个 syscall 单测

结论

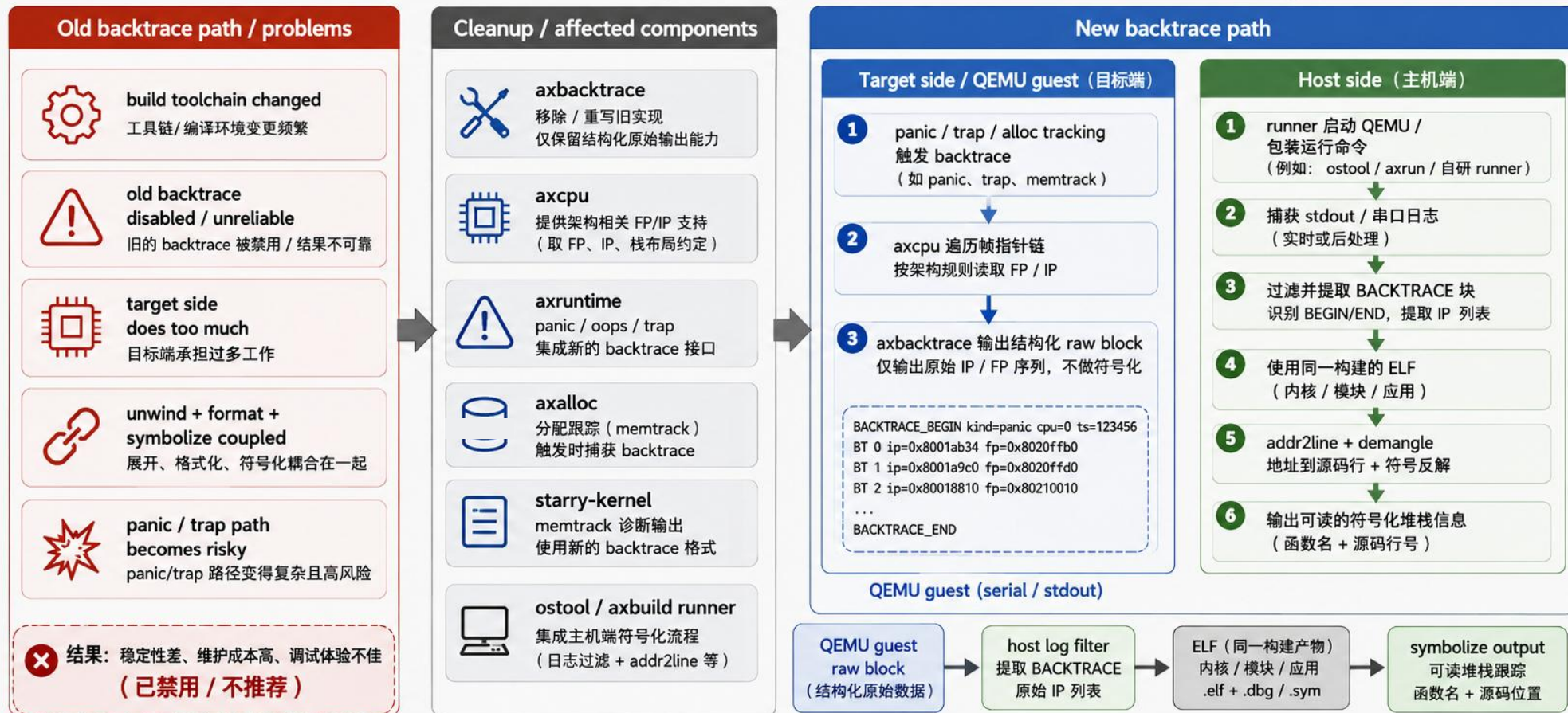
- ✅ Task2 修的是内核语义: hard link 必须共享数据状态。
- ✅ Task3 测的是真实应用行为: BusyBox link applet 能否通过 shell/libc/syscall/VFS 组合路径读到正确内容。
- ✅ 这把单点 syscall/VFS 修复转化成了长期可回归的应用级证据。

regression oracle: applet failure exposes hardlink bug

Backtrace 重做背景：从被禁用的旧实现到 host 侧诊断链路

构建工具链变化导致旧 backtrace 功能失效；重做目标不是简单恢复，而是把 target 侧 unwind 和 host 侧 symbolize 拆开，做成更稳健的诊断链路。

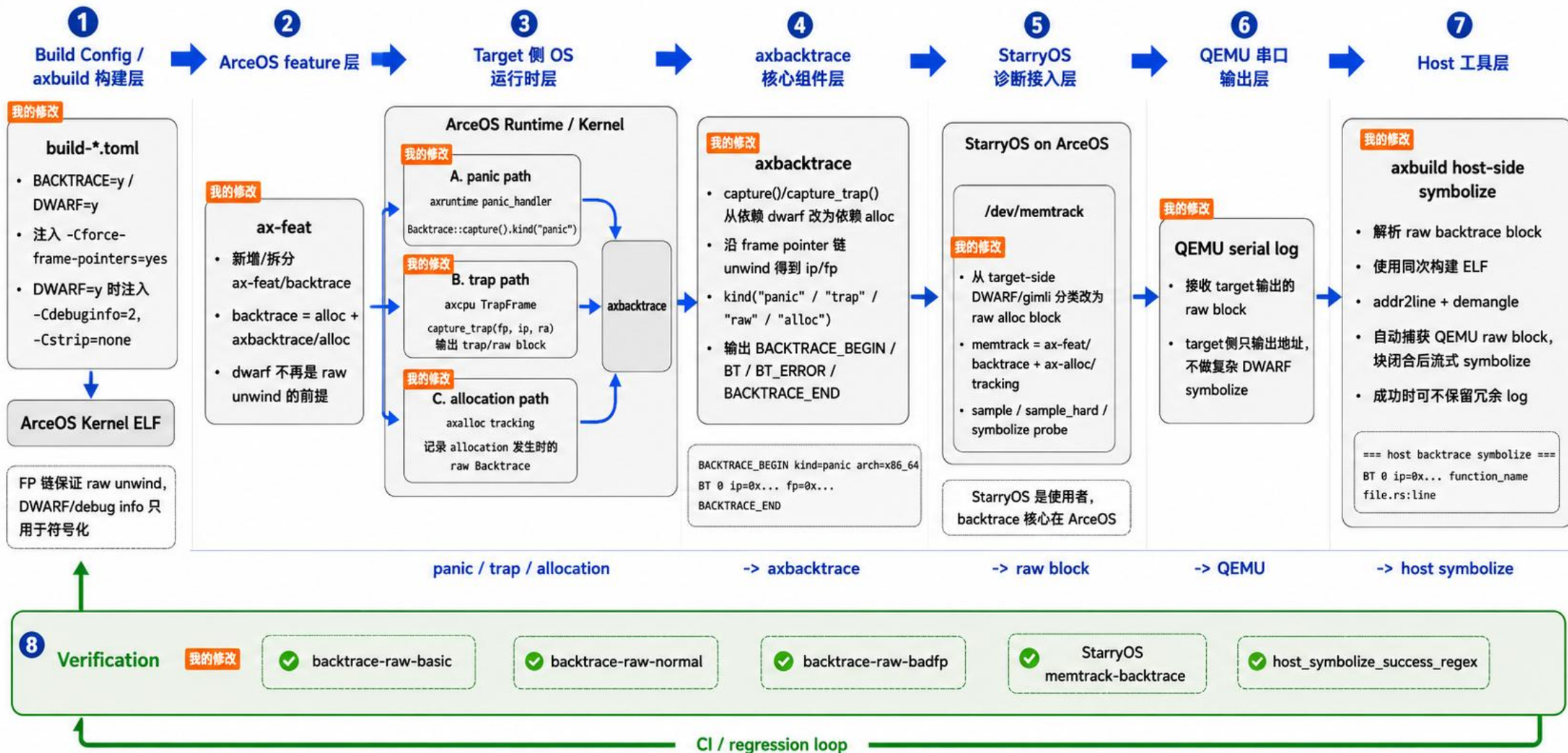
Robust Backtrace Redesign: target raw unwind + host-side symbolize



设计原则: Keep target crash paths simple; move heavy symbolization to host.
目标端仅做最小、安全、确定性的原始回溯；复杂的符号化、格式化放在主机端完成。

Backtrace 实现链路：从 OS target raw unwind 到 host-side symbolize

核心设计：target 侧负责 raw unwind，host 侧负责 symbolize；unwind 与 DWARF 符号化解耦。



Demo 1: 普通 raw backtrace (无 symbolize)

```
cargo xtask arceos test qemu \  
  --arch x86_64 --test-group rust --test-case backtrace \  
  --no-symbolize
```

Demo 2: 自动 host symbolize (函数名, 无行号)

```
cargo xtask arceos test qemu \  
  --arch x86_64 --test-group rust --test-case backtrace
```

Demo 3: DWARF 自动 symbolize

```
cargo xtask arceos test qemu \  
  --arch x86_64 --test-group rust --test-case backtrace-raw-normal
```

Demo 4: alloc tracking (memtrack)

```
cargo xtask starry test qemu \  
  --arch x86_64 --test-group normal --test-case memtrack-backtrace
```

阶段	--target	--config 的 key	flag 是否生效
#839 前	内置 triple <code>x86_64-unknown-none</code>	<code>target.x86_64-unknown-none.rustflags</code>	✅ key=triple, 匹配
#839 后 (bug)	json 路径	<code>target.'scripts/.../x86_64-unknown-none.json'.rustflags</code>	❌ key=路径串, 被静默丢弃
我的修复	json 路径 (不变)	<code>target.x86_64-unknown-none.rustflags</code> (取文件 stem)	✅ 重新匹配

DWARF=y → -Cdebuginfo=2 -Cstrip=none → 输出保留文件行号



04 未来的OS学习展望

从 BIGLAB A 看 AI 时代 OS 学习的“破与立”

韩佳辛

清华大学计算机系

2026年4月12日



目录

- 一、AI 在 OS 学习中的作用
- 二、“破”：大实验还有区分度吗？
- 三、我与 AI 的协作案例
- 四、“立”：重建学习的目标与方法



一、AI 在 OS 学习中的作用

OS

- 高度抽象：进程、虚拟内存、文件系统等概念
- 调试地狱：并发bug、指针飞跃、硬件交互，错一点就 Kernel Panic，挫败感极强

AI

- 能够反复解答知识点，直到我听懂为止
- 靠搜索引擎查文档 / 请教学长学姐 debug 的时代一去不返



二、“破”：大实验还有区分度吗？

- 现实冲击：在很多局部任务上，AI 的工程实现和查错能力已经超过了普通本科生。
- 目标错位（核心矛盾）：只要课程评价的标准还是“按时交出能通过 `make grade` 的代码”，人类趋利避害的本能，就会让我们天然倾向于依赖 AI。
- 真正的陷阱：我们面临的最严峻问题不是“不会做”，而是顺水推舟地把“理解机制、分析问题、建立脑内模型”的认知过程，连同写代码一起全盘外包给了 AI。





Starry OS 的 bug，还能修多久？

跑通一个 APP 这种事情，有多少意义？

和硬件结合！

和真实的科研场景、产业需求结合！

THANKS FOR LISTENING

韩佳辛, 2026.5.30

